Twenty-Third Implementation Iteration: RequestFactory for Request Creation

with Centralized Construction of Request Objects

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this twenty-third implementation iteration is to apply the Factory Pattern to request object creation.

Before this iteration, `DataController.cpp` was responsible for parsing request CSV rows, checking the request type, parsing time data, and directly constructing concrete request objects such as `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest`. Although that design worked, it mixed request construction responsibilities with data-loading and coordination responsibilities.

In this iteration, request creation is moved into a dedicated `RequestFactory`. The factory is responsible for creating `Request*` objects, deciding which concrete request subtype should be used, and preserving invalid or malformed request rows as `InvalidRequest` objects. As a result, `DataController` remains responsible for CSV parsing, reference lookup, and coordination, while request construction logic becomes centralized and easier to extend.

2. Changes from Iteration 22

Compared with the twenty-second iteration, the following changes were introduced:

- Added a new `RequestFactory` class
- Moved request subtype selection out of `DataController`
- Moved one-time and recurring `timeData` parsing into `RequestFactory`
- Centralized creation of:
  - `OneTimeRequest`
  - `RecurringRequest`

  - `InvalidRequest`

- Updated `DataController` so that it delegates request object creation to `RequestFactory`
- Preserved the behavior of invalid and malformed request rows by continuing to represent them as `InvalidRequest` records

This iteration does not change allocation algorithms, rule evaluation logic, user polymorphism, user creation, strategy selection, CSV file formats, or request export behavior.

3. Scenario

The prototype begins by loading users, spaces, and requests from external CSV files. `DataController` still coordinates this process. It reads the request row fields, resolves the `User*` requester and `Space*` reference, and then passes the relevant information to `RequestFactory`.

The factory receives the request identifier, request type, requester, requested space, participant count, optional fields, and raw `timeData`. Based on the request type, it decides whether to create a `OneTimeRequest`, a `RecurringRequest`, or an `InvalidRequest`. If the time data is malformed or the request type is unknown, the factory safely returns an `InvalidRequest` instead of allowing the row to disappear or cause a crash.

After request objects are created, the rest of the system continues to operate as before. Requests are evaluated by the existing rule engine and processed by the selected allocation strategy. At the end of execution, the prototype still exports:

- approved allocations to `allocations.csv`
- processed request outcomes to `request_results.csv`
- request lifecycle history together with exported request results

4. Modules and Their Tasks

A. Request Base Class Module

- Continue defining the shared interface and data of all request objects
- Continue supporting one-time, recurring, and invalid request forms

B. RequestFactory Module

- Create `Request*` objects from request row data

- Decide which concrete request subtype should be instantiated
- Parse one-time and recurring `timeData`
- Return `InvalidRequest` for malformed or unsupported request input

C. DataController Module

- Continue loading CSV row fields
- Continue resolving `User*` and `Space*` references
- Delegate request object creation to `RequestFactory`
- Continue storing all returned request objects in the system dataset

D. Rule and Strategy Modules

- Continue evaluating requests using the existing rule engine and facade
- Continue processing requests using the selected allocation strategy
- Remain unchanged by the request-creation refactor

E. Export Module

- Continue exporting approved allocations and processed request results
- Continue preserving lifecycle history and rejection reasons in exported results

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
19	Same + explicit greedy batch flow	Class + Lab + Meet	Greedy batch processing moved into strategy
20	Same + factory-based strategy selection	Class + Lab + Meet	Singleton strategy factory + interface-based selection
21	Same + priority-based processing	Class + Lab + Meet	Added `PriorityAllocationStrategy`
22	Same + polymorphic users	Class + Lab + Meet	User hierarchy + UserFactory + removal of obsolete enum role helpers
23	Same + factory-based request creation	Class + Lab + Meet	Added `RequestFactory` and removed direct request construction from `DataController`

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the new `RequestFactory` and the revised relationship between request loading and request object construction.

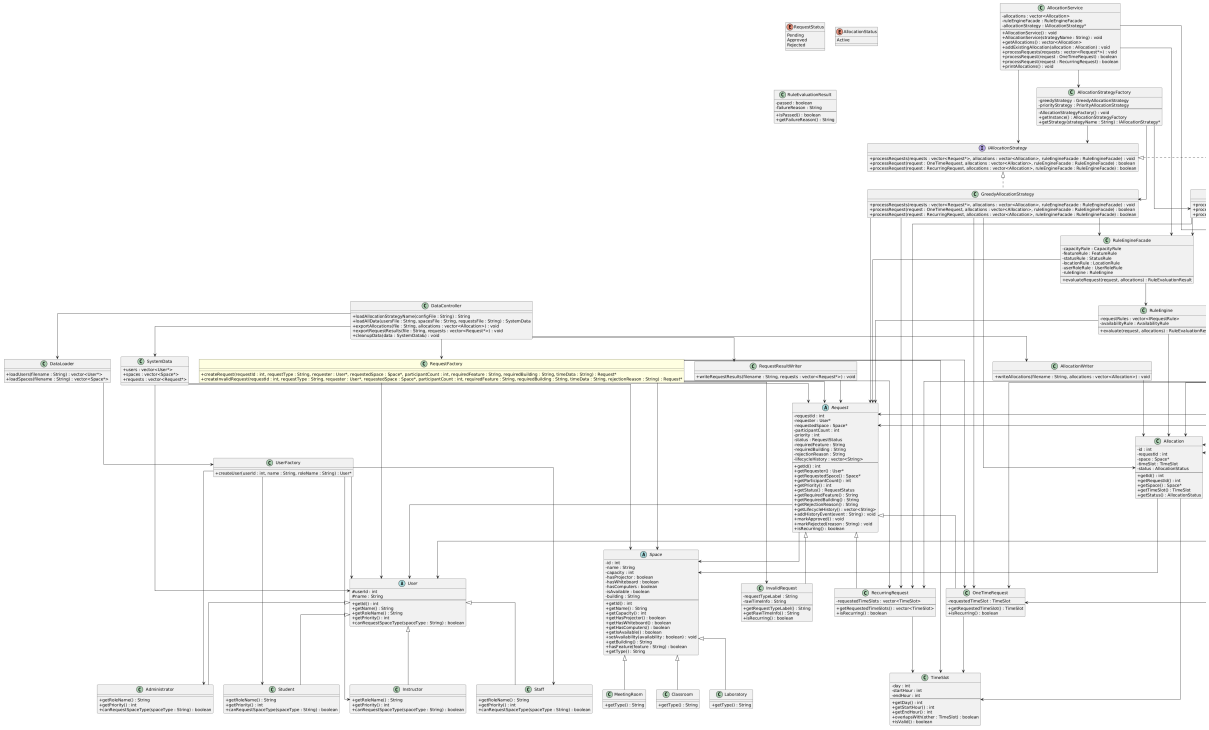

Figure 1: Updated Class Diagram with RequestFactory

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. `DataController` still parses request rows and resolves references, but the final request object construction is now delegated to `RequestFactory`.

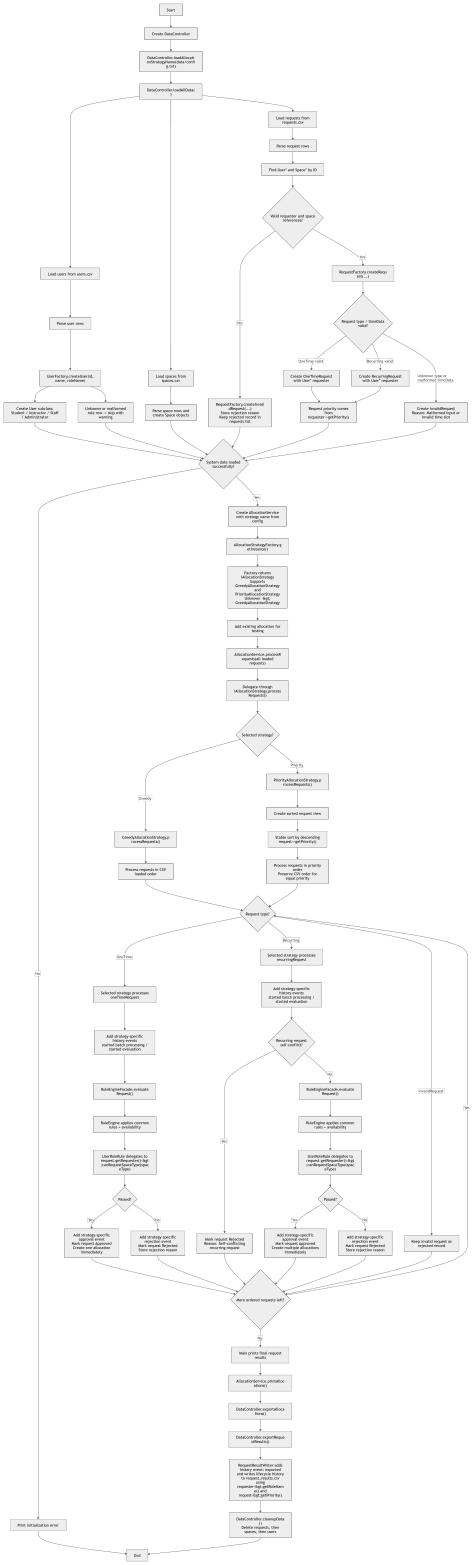

Figure 2: Workflow with RequestFactory-Based Request Creation

8. Prototype Implementation

The C++ prototype was extended by introducing `RequestFactory` as a dedicated component for request object creation. The factory receives the parsed request row information and returns a concrete `Request*` object. It supports one-time requests, recurring requests, and invalid requests within a single centralized creation path.

The request type decision and `timeData` parsing were moved out of `DataController.cpp`. If the request type is `OneTime`, the factory parses a single time slot and creates a `OneTimeRequest`. If the request type is `Recurring`, the factory parses multiple time slots and creates a `RecurringRequest`. If the request type is unknown or if time parsing fails, the factory returns an `InvalidRequest` with the corresponding rejection reason.

This design keeps responsibilities cleaner. `RequestFactory` is responsible only for request object creation. It does not evaluate rules, does not allocate spaces, and does not choose allocation strategies. `DataController` remains responsible for loading CSV fields, finding user and space references, and coordinating the creation and storage of system data.

The rest of the prototype remains unchanged. Request evaluation, allocation strategies, user creation, strategy selection, CSV exports, and request history handling continue to operate through the same architecture as before.

9. Summary

This twenty-third implementation iteration applies the Factory Pattern to request creation by introducing `RequestFactory`. The new factory centralizes construction of `OneTimeRequest`, `RecurringRequest`, and `InvalidRequest` objects, while keeping malformed and unsupported request input safe and visible in the system.

This change improves separation of responsibilities by removing concrete request construction logic from `DataController`. As a result, the prototype becomes easier to extend with future request types while preserving the current rule engine, strategy behavior, CSV format, lifecycle history, and export workflow.